

Python Weekly Assignment

Topics: Exception Handling · NumPy · Pandas Basics

Section A — Exception Handling

1. Write a function `safe_divide(a, b)` that divides two numbers and handles `ZeroDivisionError` and `TypeError` separately, printing an appropriate message for each. Use a `finally` block to print 'Operation attempted.' regardless of the outcome. Test it with valid numbers, division by zero, and a string input.
2. Write a function `validate_age(age)` that raises a custom exception `InvalidAgeError` (a class you define, inheriting from `Exception`) if age is negative or greater than 120. Catch this exception in a calling block and print a user-friendly message. Test it with at least 3 different values.
3. Write a program that repeatedly asks the user (use a loop with at least 3 simulated inputs in a list, e.g. `user_inputs = ['10', 'abc', '0', '5']`) to enter a number and divide 100 by it. Handle the following in a single `try` block using multiple `except` clauses:
 - a) `ValueError` if the input is not a number.
 - b) `ZeroDivisionError` if the input is zero.
 - c) A generic `Exception` as a catch-all for anything else.

Print the result if successful, otherwise print which error occurred.

Section B — NumPy

4. Create a 1D NumPy array of integers from 1 to 20. Perform the following:
 - a) Reshape it into a 4x5 2D array.
 - b) Print the sum of each row and each column.
 - c) Extract all elements greater than 10 using boolean indexing.
 - d) Replace all even numbers in the array with -1.
5. Create two NumPy arrays A and B as shown below and perform:

```
A = np.array([[1, 2], [3, 4]]) B = np.array([[5, 6], [7, 8]])
```

 - a) Element-wise addition, subtraction, and multiplication.
 - b) Matrix multiplication using `np.dot()` or the `@` operator.
 - c) Transpose of A.
 - d) The determinant and inverse of A using `np.linalg`.
6. Generate a NumPy array of 15 random integers between 1 and 100 using `np.random.randint()` (set a seed of 42 for reproducibility). Then:
 - a) Find the mean, median, standard deviation, min, and max.
 - b) Sort the array in descending order.
 - c) Find the indices of the 3 largest values using `np.argsort()`.
 - d) Normalize the array so all values lie between 0 and 1.

Section C — Pandas Basics

7. Create a Pandas DataFrame from the dictionary below and perform the following:

```
data = { "Name": ["Aarav", "Priya", "Karan", "Meera", "Sahil"], "Age": [23, 21, 25, 22, 24], "Department": ["AI", "Data Science", "AI", "Web Dev", "Data Science"], "Score": [88, 92, 79, 95, 67], }
```

- Display the first 3 rows and the data types of each column.
- Add a new column Grade based on Score (≥ 90 'A', ≥ 75 'B', else 'C').
- Sort the DataFrame by Score in descending order.
- Filter rows where Department is 'AI'.

8. Using the same DataFrame from Q7:

- Group by Department and find the average Score for each.
- Find the row with the highest Score using `idxmax()`.
- Rename the column Score to Marks.
- Drop the Age column and display the resulting DataFrame.

9. Create a DataFrame with some missing values as shown below and handle them:

```
import numpy as np data = { "Product": ["Pen", "Notebook", "Eraser", "Sharpener", "Ruler"], "Price": [10, np.nan, 5, 8, np.nan], "Stock": [100, 50, np.nan, 30, 20], }
```

- Check the number of missing values in each column using `isnull().sum()`.
- Fill missing Price values with the column mean.
- Drop rows where Stock is still missing.
- Save the cleaned DataFrame to a CSV file named `cleaned_products.csv` without the index.

10. Read the CSV file created in Q9 (`cleaned_products.csv`) back into a new DataFrame and:

- Print its shape and column names.
- Add a new column `Total_Value` as Price multiplied by Stock.
- Use `describe()` to print summary statistics of the numeric columns.
- Wrap the file reading step in a try-except block to handle the case where the file does not exist (`FileNotFoundException`).